

Arete - The Chess Tutor That Does Not Try to Win

Frequently Asked Questions

STA561D: Probabilistic Machine Learning | Duke University | Spring 2026

System Design

Q1: What does Arete do that a chess engine does not?

A chess engine optimises for winning. Arete optimises for teaching. The best move for a computer is rarely the best move for a learner, not because the learner cannot play it, but because they cannot learn from it. Arete recommends the move a player at a specific skill level would actually find and understand, paired with an explanation written in language matched to where they are. The system sits at the human-algorithm interface rather than replacing either the engine or the learner.

Three things a player immediately notices. First, it speaks their language: a beginner gets plain English, a concrete warning, and a one-move plan; an advanced player gets technical vocabulary, a five-move strategic plan, and a comparison of the strongest alternatives. Second, it learns who they are without requiring a self-reported rating: ten feedback responses is enough for the system to identify skill level reliably. Third, it tells the player when to be sceptical of its own recommendation: both the complexity meter and the confidence indicator flag positions where the recommendation is less certain.

Q2: Why Stockfish Skill Level rather than a purpose-trained model?

Stockfish's Skill Level parameter degrades play in a controlled way by introducing stochastic move selection. At full strength the engine almost always picks the same best move. At low skill it picks from a broader distribution of moves with similar frequency, mimicking the less decisive play of a weaker player. Training a custom model to generate ELO-appropriate moves would require a large, labelled dataset and significant compute.

The framing is behavioural cloning: the system imitates the distribution of moves observed at a given ELO band rather than optimising for win probability. The temperature analysis experiment confirms this interpretation. Shannon entropy of the move distribution decreases consistently with Skill Level with Spearman correlation of negative 0.905 and p-value of 0.002, confirming that Skill Level functions as an implicit temperature parameter in a Boltzmann distribution over moves. The more rigorous implementation, Maia Chess trained on twelve million Lichess games stratified by ELO, is the identified upgrade path.

Q3: What are the three modes?

Tutor Mode allows the player to analyse any position and receive an ELO-calibrated move recommendation with a structured three-section explanation. The player can play against themselves with full analysis on every move or load any custom position via FEN string. Play vs Bot allows the player to play a full game against Stockfish calibrated to their ELO, with commentary after each bot move. Endgame Trainer is a King and Pawn vs King endgame on a five-by-five board, where the opponent is a reinforcement learning agent that learned through trial and error across five hundred thousand games with no prior knowledge of chess strategy.

Probabilistic Components

Q4: What is the Bayesian ELO estimator and why is it the right approach?

The Bayesian ELO estimator treats the player's true skill level as a latent variable and maintains a posterior probability distribution over nine ELO bands. After each explanation, the player rates it as Too Simple, Right Level, or Too Complex. A Gaussian likelihood model characterises how each feedback signal relates to the gap between true and displayed ELO. The posterior updates via Bayes' rule after every observation.

This is the principled statistical solution to inferring a latent quantity from noisy sequential observations. It produces a calibrated uncertainty estimate alongside the point estimate: the posterior standard deviation tells the system how confident it is in its current ELO inference before acting on it. Validated by simulation: every simulated player was identified within one ELO band after twenty feedback rounds, with posterior standard deviation dropping from 516 to approximately 104 ELO points, an eighty percent reduction in uncertainty. Optimal likelihood width is 300 ELO points from sensitivity analysis across five values.

Q5: What does the position complexity meter measure?

It measures the Shannon entropy of Stockfish's move selection distribution. The system samples Stockfish fifteen times from the current position and computes H equals negative sum of $p \log p$ over all sampled moves. High entropy means many moves are selected with similar frequency, the position has genuine move choice and is intrinsically complex. Low entropy means one move dominates, the position is clear and the engine agrees with itself.

This directly affects explanation reliability. The faithfulness experiment showed that explanation accuracy in complex positions drops measurably, with the most striking case being ELO 1800 in a sharp middlegame where faithfulness fell to 0.43 as strategic abstraction overtook positional specificity. The meter gives the player an honest signal about when to expect this, before they read the explanation.

Q6: What does the evaluation confidence indicator measure?

It compares Stockfish centipawn evaluation at search depth 5 versus depth 15. The gap delta equals the absolute difference between the two evaluations in pawn units. A large gap means the engine revises its assessment significantly as it calculates further ahead, indicating tactical complications that only emerge with deeper analysis. A small gap means shallow and deep analysis agree and the recommendation is well grounded.

The depth-variance experiment confirmed this across three position types. The quiet opening stabilised quickly with low standard deviation across nine tested depths. The sharp tactical position showed the largest swings, with the assessment changing meaningfully as search depth increased. Thresholds: High confidence below 0.3 pawns, Medium between 0.3 and 0.8 pawns, Low above 0.8 pawns.

Q7: How does the reinforcement learning endgame trainer work?

The endgame trainer uses a Monte Carlo Control agent trained on five hundred thousand King and Pawn endgame episodes with randomised starting positions.

The algorithm is first-visit Monte Carlo Control with epsilon-greedy exploration. The Q-value update rule is:

$$Q(s, a) \leftarrow Q(s, a) + [1 / N(s, a)] \times [G - Q(s, a)]$$

where G is the return from state s , $N(s, a)$ is the visit count for that state-action pair, and the epsilon schedule is:

$$\varepsilon(s) = N_0 / (N_0 + N(s)) \text{ with } N_0 = 100$$

Randomised starting positions are critical. A fixed starting position produces an agent that memorises one trajectory, visiting only twenty states. Randomised positions require the agent to learn a general policy, visiting two thousand one hundred and thirty-eight distinct states across training. The agent achieved a 97.6 percent win rate after five hundred thousand episodes and generalised correctly to board configurations it had never encountered during training, including cases requiring king coordination before pawn advancement.

The trainer connects directly to the broader system: the same learning algorithm that handles this small endgame underpins full-scale chess AI. What changes at larger scale is the representation, a neural network rather than a lookup table, and the search procedure, MCTS rather than epsilon-greedy. The Bellman equation and policy improvement through experience are identical.

Experimental Results

Q8: How do you know the explanations are calibrated to ELO?

Two measurements at two levels. Surface calibration is confirmed: Flesch-Kincaid Grade Level increases monotonically from 6.8 at ELO 800 to 11.4 at ELO 1800, a span of 4.6 grade levels. Flesch Reading Ease drops from 73.5 to 32.9. Advanced vocabulary usage increases from zero terms at ELO 800 to five at ELO 1800.

Semantic calibration is weaker. The ELO recovery experiment fed explanations back to the model as a classifier without ELO labels. Accuracy was 25%, with the classifier predicting ELO 1600 for three of four inputs regardless of true level. Vocabulary constraint enforcement works at the word level but conceptual depth does not scale as cleanly. This is the primary identified limitation and the central motivation for the Maia Chess upgrade path.

Q9: Which prompt engineering strategy works best?

No single strategy dominated on all three calibration quality dimensions tested. Structured prompting produced the widest Flesch-Kincaid range across ELO bands at 2.79 grade levels. Role-play prompting achieved better vocabulary suppression at low ELO with zero advanced terms versus 0.5 for Structured. All four strategies achieved identical monotonicity scores of 0.67, meaning none perfectly scaled complexity at every ELO transition. A hybrid approach combining Structured's calibration range with Role-play's vocabulary compliance is the recommended next design iteration.

Q10: What did the strategy-ELO interaction experiment find?

TF-IDF cosine similarity between Aggressive and Positional explanations was computed at each ELO level. Similarity ranged from 0.129 to 0.179, with peak differentiation at ELO 1600 rather than ELO 1800 as hypothesised. The expected interaction effect, that strategy differentiation increases as vocabulary constraints relax at higher ELO, was not confirmed. Style injection produces consistent variation across ELO bands rather than ELO-dependent variation. A more integrated implementation would bias Stockfish's move selection by style rather than affecting only the explanation framing.

Differentiators and Future Work

Q11: What makes Arete different from existing chess tutoring tools?

Three properties are genuinely distinctive. First, it is the only tutoring system that maintains a posterior probability distribution over player ELO and updates it from explanation comprehension signals rather than requiring self-reported ratings or inferring from game outcomes. Second, it exposes position uncertainty through two independent measures, entropy of the move distribution and depth-variance of the evaluation, giving

the player calibrated signals about when the recommendation is reliable. Third, the explanation layer is architecturally separated from the move generation layer, allowing independent improvement of each without rebuilding the system.

Q12: Is this reinforcement learning?

The move generation and explanation components are not RL. Move generation is behavioural cloning and explanation generation is prompt engineering on a pre-trained language model. The Bayesian ELO estimator is online Bayesian inference. The endgame trainer is RL, implementing Monte Carlo Control to learn a general King and Pawn endgame policy from experience.

Full RL-based move generation is the long-term vision. The endgame trainer is the proof of concept. Scaling it to full chess requires a neural network Q-function, TD-learning rather than Monte Carlo returns, and MCTS rather than epsilon-greedy exploration. Maia Chess is the practical near-term step, using imitation learning on human game data to produce ELO-appropriate moves before RL fine-tuning becomes feasible.

Q13: What are the main limitations?

- Semantic calibration gap: ELO recovery accuracy 25% despite strong surface readability metrics
- No single prompt strategy dominates on all calibration dimensions: hybrid approach is the recommended next step
- High-ELO faithfulness degradation: ELO 1800 middlegame faithfulness 0.43, abstraction overtakes positional specificity
- Strategy-ELO interaction null result: style differentiation does not increase with ELO as hypothesised
- Move input uses square-selector dropdowns rather than click-to-move board interaction
- No persistent session memory: system does not track mistake patterns across games

Q14: What is the cost of running this system?

All LLM calls use Claude Haiku, Anthropic's most efficient model. Each explanation call consumes approximately 800 tokens at under one cent per call. The full development, testing, and seven-experiment evaluation cycle cost approximately fifteen to twenty dollars in API credits over a period of time (an approximate attribution because I had other overlapping tasks also). Stockfish 18 is free and open source and runs locally. A typical demo session of twenty moves costs under twenty cents.